

	Departamento de Engenharia Eletrónica e Telecomunicações e de Computadores Computação Distribuída (Época Normal) MEIC, MEIM	
Nº	Nome:	13/01/2023

EXAME ÉPOCA NORMAL (1ª PARTE, SEM CONSULTA, DURAÇÃO: 45 min., 11 valores)

Nas questões de 1 a 5 assinale as afirmações como verdadeira (V), falsa (F) ou não responde (—). Uma opção assinalada corretamente soma 0,25 valores e incorretamente desconta 0,125 valores (50% da cotação de cada alínea).

1. [1 val.] Sobre as características dos sistemas distribuídos

☐	O teorema CAP demonstra que a escalabilidade de um sistema depende da capacidade de CPU
☐	O desacoplamento (<i>loosely coupled</i>), entre as partes, depende da existência de contratos, por exemplo contratos <i>protobuf</i> em gRPC, para separar as implementações do cliente e do servidor
☐	Um <i>stub</i> de cliente em gRPC, bloqueante ou não bloqueante facilita a total transparência à localização que é uma das características desejáveis nos sistemas distribuídos
☐	A Latência é um fator irrelevante podendo ser zero, em aplicações distribuídas

2. [1 val.] Sobre o conceito de Tempo e Coordenação em sistemas distribuídos

☐	Os valores de um relógio físico (hh:mm:ss) podem ser sincronizados entre múltiplos computadores usando o protocolo <i>Network Time Protocol (NTP)</i>
☐	Devido à inexistência de um relógio físico global, exatamente igual em todos os computadores, a ordenação de eventos tem de recorrer a relógios lógicos como proposto por Leslie Lamport em 1978
☐	Os relógios lógicos como proposto por Leslie Lamport em 1978 permitem ordenar os eventos e detetar dependências causais entre eventos num sistema distribuído
☐	Os relógios lógicos vetoriais transportam nas mensagens valores dos relógios entre participantes que permitem detetar dependências causais (a ocorrência de um evento ser consequência de outros)

3. [1 val.] Relativamente ao desenvolvimento de aplicações em gRPC

☐	Quando numa operação <i>rpc</i> se usa <i>streaming</i> de cliente e de servidor, a cada mensagem do cliente tem de corresponder uma mensagem de resposta do servidor
☐	Uma operação que usa <i>streaming</i> de cliente e de servidor não pode ser chamada com um <i>stub</i> bloqueante
☐	Numa operação com <i>stream</i> de cliente, o cliente tem a garantia de que as mensagens chegam ao servidor pela ordem com que foram enviadas
☐	Um servidor gRPC pode implementar diferentes contratos <i>protobuf</i> para diferentes tipos de cliente

4. [1 val.] Sobre o sistema RabbitMQ

☐	Para evitar que se percam mensagens enviadas com <i>routing keys</i> inválidas um <i>exchange</i> pode ter associado um <i>alternate-exchange</i>
☐	Sobre uma fila (<i>queue</i>) pode usar-se a política de <i>work-queue</i> para certificar que todos os consumidores desse <i>queue</i> recebem todas as mesmas mensagens
☐	Quando um consumidor dá <i>acknowledge</i> negativo a uma mensagem o produtor da mensagem não recebe nenhuma notificação
☐	Quando um consumidor dá <i>acknowledge</i> negativo a uma mensagem a mesma é descartada, não sendo recebida posteriormente por qualquer outro consumidor

5. [1 val.] Sobre algoritmos de exclusão mútua, eleição e consensos

☐	O algoritmo de exclusão mútua <i>Ricart&Agrawala</i> que envolve um número elevado (N) de participantes é pouco eficiente pois cada autorização de acesso requer $2 \cdot (N-1)$ mensagens
☐	No algoritmo <i>Raft</i> o consenso é garantido pela eleição de um servidor líder, que tem a responsabilidade de coordenar a atualização das réplicas dos <i>logs</i> que servem de base às atualizações de dados
☐	O protocolo <i>Two-phase commit (2PC)</i> é muito utilizado na coordenação de consensos pois é totalmente tolerante a falhas do coordenador ou de algum dos participantes
☐	O algoritmo de eleição <i>Bully</i> utiliza relógios lógicos, sendo eleito o coordenador que tiver o menor relógio

6. [2 val.] Considere um sistema distribuído, constituído por 3 participantes (**p0**, **p1**, **p2**), que utiliza comunicação por *multicast* fiável e um mecanismo baseado em relógios vetoriais que garante ordenação causal de mensagens. Quando **p0 envia a mensagem** com o vetor **[5,2,6]**, justifique a razão porque acontece o seguinte nos outros participantes **p1** e **p2**:
- a) Como **p1** tem o seu vetor local **p1=[3,3,5]**, em **p1** a mensagem vinda de **p0** é retida (*Hold*);
 - b) Como **p2** tem o seu vetor local **p2=[4,1,6]**, em **p2** a mensagem vinda de **p0** é retida (*Hold*);

7. [2 val.] Considere que usa o *middleware* de comunicação por grupos (*Spread Toolkit*) no desenvolvimento de uma aplicação Cliente/Servidor, onde se pretende ter um grupo de múltiplos servidores, em que cada um trata o mesmo pedido **P** com algoritmos diferentes e o cliente obtém as múltiplas respostas **R1,...,RN**, podendo escolher as que considerar mais adequadas. Desenhe um esquema de arquitetura e descreva com frases simples como implementava o sistema.

8. [2 val.] Considere uma imagem Docker com uma aplicação servidora gRPC que aceita conexões clientes através do porto 8000 e que escreve mensagens de *logging* no ficheiro `/user/app/Logs.txt`. Considerando que só tem um computador (*host*) com suporte para Docker, descreva como executar nesse computador *host* múltiplos servidores gRPC (**app1,...,appN**), em que todos eles escreveriam os *logs* no computador *host* em ficheiros com os nomes `/usr/app1/Logs.txt, ...,/usr/appN/Logs.txt`.